

Labo 3 - Partitioning the grid into warps and reducing thread divergence

Robin De Vos (MEIECFS)
Anthony Tacquet (MEISWE)

Februari 2026

1 Context

1.1 Part 1

For part 1 of this lab exercise we had to implement a CUDA functionality that would calculate the inverse value of each RGB colour within each pixel according to the formula below.

$$r[idx] = 255 - a[idx]$$

Where current `idx` is the index of a colour value, no divergence will occur in this scenario since each colour value is calculated in the same manner.

1.2 Part 2

For this assignment we were asked to maintain the same code for inverting the green and blue values of each pixels, but instead of inverting the red value we had to calculate the 1D-Gaussian blur, which takes into account its neighbouring red values.

2 Methodology

For both Part 1 and 2 of this lab exercise we used the strides technique, this way one thread has to sequentially calculate the inverted colour value, depending on the Grid and Block Dimension the amount of strides is calculated. Logically the more Threads one has the less strides are needed, meaning threads have to calculate a lesser amount of colour values. This applies for Blocks as well. The way we calculate strides can be seen below.

$$strides = \frac{array_length}{blockDim.x * gridDim.x}$$

Where `array_length` is the length of array `a` and `r`. Array `a` contains all colour values (red, green and blue) and array `r` is used to store the result of the inversion calculation and Gaussian calculation used in Part 2.

2.1 Part 1

The code in listing 1 is straight forward, we already explained in section 2 how strides gets calculated, what we also use is the variable `array_index` which can be seen as the corrected version of the core index (that includes the strides offset).

Listing 1: Color inversion code snippet

```
1 __global__ void invert(uint8_t *a, uint8_t *r, int n) {
2         int idx = threadIdx.x + blockIdx.x * blockDim.x;
```

```
3         int strides = n / (blockDim.x * gridDim.x);
4
5         for (int i = 0; i < strides; i++) {
6                 int array_index = idx + (i * blockDim.x);
7
8                 if (array_index < n) {
9                         r[array_index] = 255 - a[array_index];
10                }
11        }
12}
```

2.2 Part 2

In listing 2 and 3 we use largely the same code but instead of just inverting we create an if statement that checks if the `array_index` points to a red value or not. The difference between listing 2 and 3 is that we either check if the index is exactly dividable by three (first, third, sixth, ... values are red) or whether we are operating in the first third of the array (if the array is ordered RRR...GGG...BBB...).

You might have noticed we use an offset of either 6 or 2 if the array is ordered in order to exclude the first two pixels ($2 * 3 = 6$) or first two red values respectively. This is because we wanted to keep our code simple and thus excluded these edge cases.

Listing 2: Color inversion and Gaussian blur code snippet

```
1 __global__ void invert(uint8_t *a, uint8_t *r, int n) {
2         int idx = threadIdx.x + blockIdx.x * blockDim.x;
3         //int strides = n / MAX_THREADS_PER_BLOCK;
4         int strides = n / (blockDim.x * gridDim.x);
5
6
7         for (int i = 0; i < strides; i++) {
8                 int array_index = idx + (i * blockDim.x);
9
10                if (array_index < n) {
11                        // Lazy for the edge cases for the first
                             two and last two pixels
12                        if (array_index % 3 == 0 && array_index >
                              6 && array_index < n - 6) {
13                            r[array_index] = 0.1*a[
                                 array_index - 6] +  0.25 * a[
                                 array_index - 3] + 0.5 * a[
                                 array_index] + 0.25 * a[
                                 array_index + 3] + 0.1 * a[
```

```
                                        array_index + 6];
14                          } else {
15                                  r[array_index] = 255 - a[
                                        array_index];
16                          }
17                  }
18          }
19  }
```

Listing 3: Color inversion and Gaussian blur code snippet using transformed pixel array

```
1   __global__ void invert(uint8_t *a, uint8_t *r, int n) {
2           int idx = threadIdx.x + blockIdx.x * blockDim.x;
3           int strides = n / (blockDim.x * gridDim.x);
4
5           for (int i = 0; i < strides; i++) {
6                   int array_index = idx + (i * blockDim.x);
7
8                   if (array_index > 2 && array_index < (n / 3) - 2)
                        {
9                           r[array_index] = 0.1*a[array_index - 2] +
                                0.25 * a[array_index - 1] + 0.5 * a[
                               array_index] + 0.25 * a[array_index +
                               1] + 0.1 * a[array_index + 2];
10                  } else {
11                          r[array_index] = 255 - a[array_index];
12                  }
13          }
14  }
```

3 Observations/results

Our inversion Gaussian functions does not calculate the red value for the first and last two pixels in the 1D array, since that would create an out of bounds reference (if no checks are implemented). We decided to ignore these edge cases, an alternative approach could have included the Gaussian formula to either use zero or its own value for four pixels at the beginning and end of the array. We decided not to include these scenarios since the extra if statements and check required to validate and compute these extra 4 red value out of the 409600 red values would be overkill.

Another aspect we did not yet touch was the fact that the Gaussian Blur at the edges of the image uses the red values from the opposite side of the image in its calculation since

the data is stored in a 1D array. We didn't take this into account neither since this was out of the scope of this lab.

3.1 Part 1

It's difficult to visualise whether to increase the block size or the thread size if you require a certain amount of threads for your specific application. We tried to plot this in the best way possible and what you can see in figure 1 is a plot which has as y-axis the duration of the total inversion calculation and on the x-axis the total threads we gave our GPU to perform this calculation. The interesting part about these plots is how we calculate the amount of threads the GPU can use.

16 threads with increasing blocks (half-warp):

What we do here is take a fixed amount of threads (we chose 16) per block, if you remember that a warp consists of 32 threads this means that half of these threads within this warp will remain 'idle'. We also take a variable amount of blocks, initially this value will be 2, but this value will double for each iteration performed. In total we will 'allocate' $16 * 2$ useful threads which equals 32 threads initially. This iterative process can be seen in table 1.

Blocks	Threads	Total Threads
2	16	32
4	16	64
8	16	128
16	16	256
32	16	512
64	16	1024
128	16	2048
...	...	...

Table 1: Half-warp thread configuration

32 threads with increasing multiple of 32 and eventually increasing block size:

In this plot we use 32 threads per block, but we double the amount of threads until we reach 1024 threads, then we maintain this value and start increasing the block count, this can be seen in table 2.

Blocks	Threads	Total Threads
1	32	32
1	64	64
1	128	128
1	256	256
1	512	512
1	1024	1024
2	1024	2048
...	...	...

Table 2: Full-warp thread configuration

We can clearly see that for each value of x both plots have the exact same amount of threads but a different duration. We can determine from figure 1 that using more blocks instead of threads does increase performance, the blue area shown in the graph illustrates the moment our full-warp graph start using multiple blocks as well, shortly after that we can actually see that the full-warp plot overtakes the half-warp plot in performance. At a certain minimum we can see that the scheduling overhead most likely creates a drag in performance and the duration actually increases.

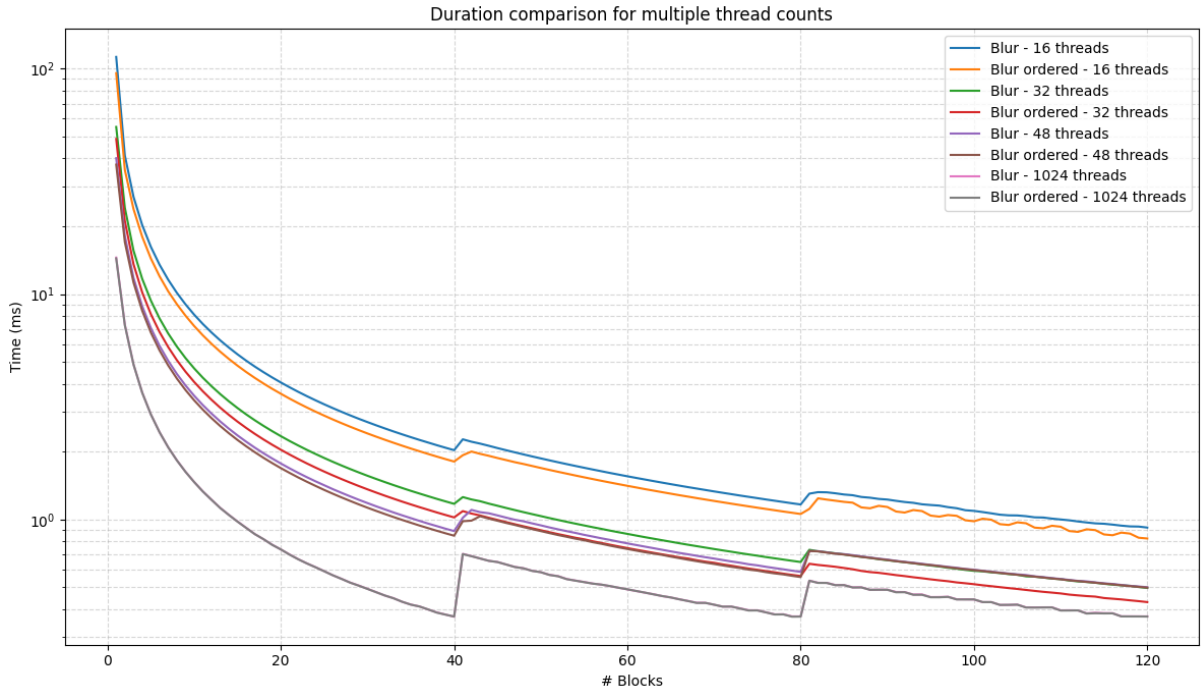


Figure 1: Graph comparing half-warp and full-warp with increasing block size

3.2 Part 2

In figure 2 its clear that there is a processing speed up when the one dimensional colour array gets ordered beforehand, this way the cores in the GPU can all first focus on the more difficult red value calculation and afterwards perform the easier inversion calculation. This is called divergence, this problem is solved when using the ordered array since cores do not stay idle for long when all perform roughly the same calculation.

We can also see that certain performance loss occurs when we reach a block size that is a multiple of 40, this happens at 40 and 80 blocks and perhaps even further. This has probably something to do with the actual hardware limits of our GPU.

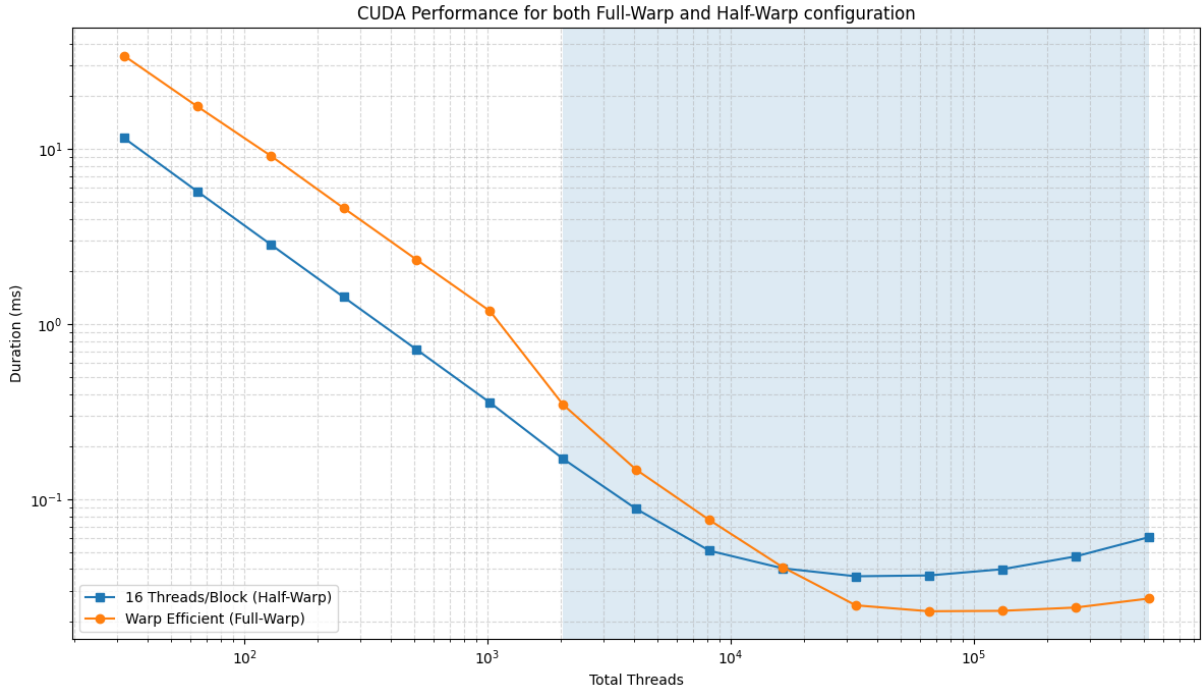
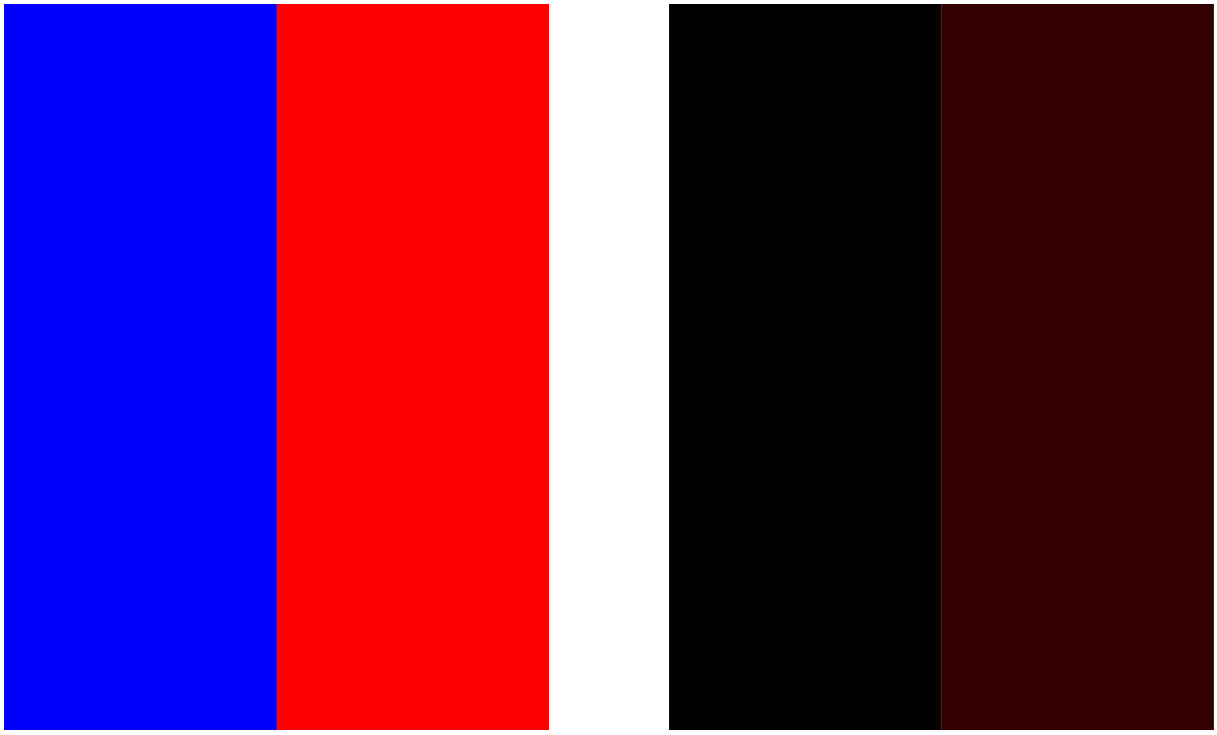


Figure 2: Graph of duration comparison between ordered array and unordered array

We made an additional test where we created a .ppm image that has a 0,0,255 (pure blue) colour on the left and a 0,0,255 (pure red) colour on the right. The result of this experiment can be seen in figure 3. Because we used one long 1D array where the rightmost pixel of row N gets placed right before the leftmost pixel of row N + 1. This results in the Gaussian blur to generate inaccurate red values at places where no red may even be close by. This can be seen in figure 3b (might not be clear in this document), on the left side of the image there is a vertical red coloured line across the whole length of the image.

Somehow the centreline of the output image seems to have gotten a higher red value even though the calculation for these pixels largely depends on the value of its right sided neighbours.

A clearer image of the left sided part of figure 3b can be found in figure 5 in the Appendix.



(a) Red blue image

(b) Red blue image after Gaussian blur

Figure 3: Red blue split image

4 Conclusion

As you can see from figure 4, the results look to be correct, the inverted Gaussian Anna is identical for both our ordered and unordered one dimensional array.

When doing our measurements for the half- and full-warp configuration we expected the half-warp performance to be worse than the full-warp performance, this seemed not to be the case, it seems that using more blocks in this scenario gives a greater performance boost.

The issues with the one dimensional array that we discussed earlier does not have any significant effect on the output, these effects are almost unnoticeable in normal images which can be determined from figure 4c.

During programming we encountered some issues when calculating the strides, at first we used $strides = n/1024$ this was supposed to be a temporary solution since we decided to test our code first with a single block and 1024 threads, which is correct in that scenario. We forgot to change this equation afterwards and got some weird timing related issues, but was luckily fairly easy to resolve.



(a) Original Anna (b) Inverted Anna (c) Inverted Gaussian Anna

Figure 4: Images of Anna used in CUDA

5 Appendix

Note: the code provided below also contains our timing code as well as the code used to make our csv data needed for our visualisations.

5.1 Images

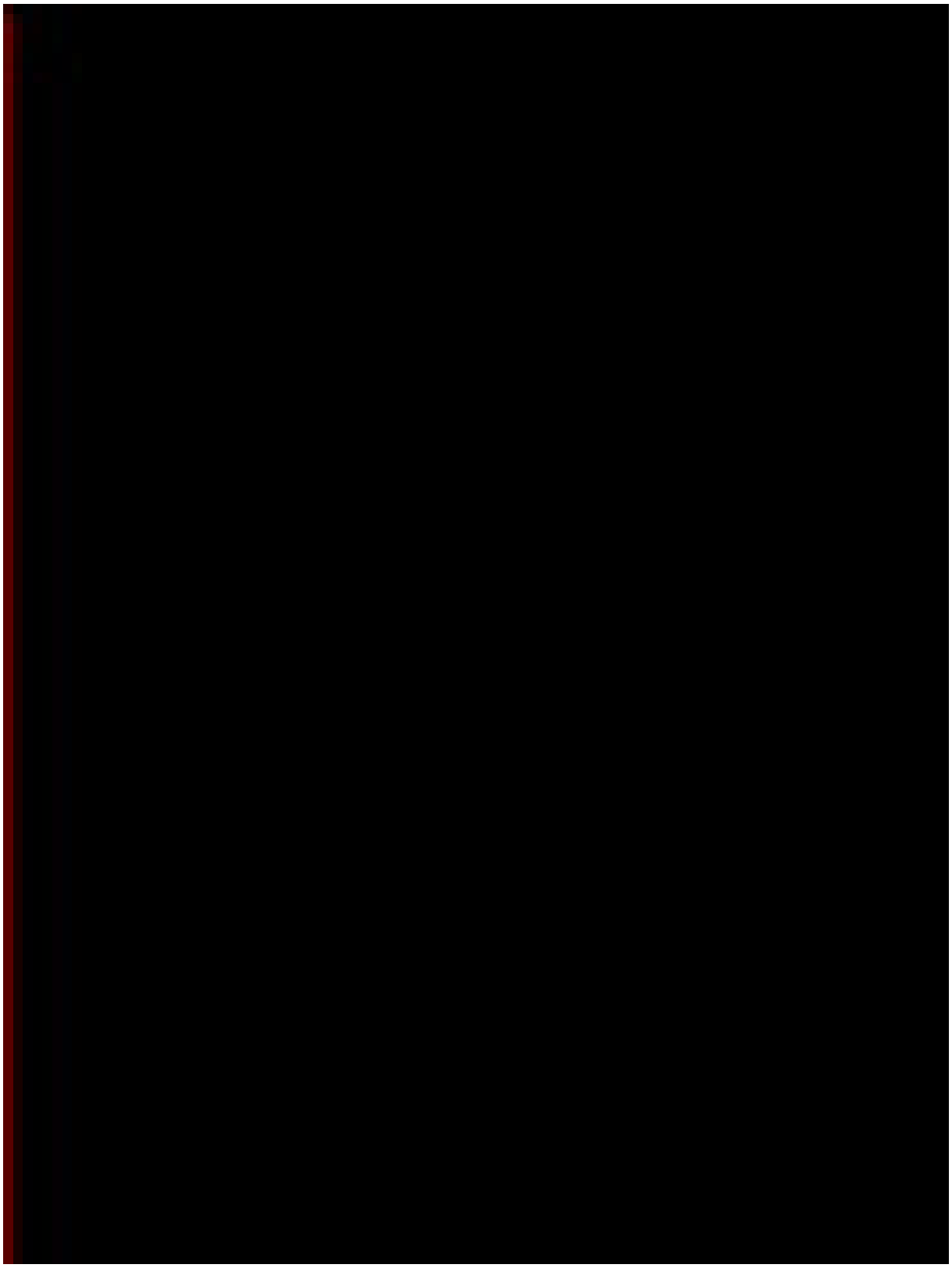


Figure 5: Cropped image of the red coloured edge of the red blue image

5.2 Code Part 1

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run 'convert <name>.<
    extension> <name>.ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <fstream>

// #define M 512        // Lenna width
// #define N 512        // Lenna height
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length

__global__ void invert(uint8_t *a, uint8_t *r, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    //int strides = n / 1024;;
    int strides = n / (blockDim.x * gridDim.x);

    for (int i = 0; i < strides; i++) {
        int array_index = idx + (i * blockDim.x);

        if (array_index < n) {
            // Lazy fuck the edge cases for the first two and
               last two pixels
            r[array_index] = 255 - a[array_index];
        }
    }
}

uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
```

```
41      *   - The first 3 data elements are the RGB components
42      *   - The first 3*M data elements represent the firts row of
            the image
43      *   - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M},
            r_{1,0}, ..., b_{b,M}, ..., b_{N,M}
44      *
45      */
46     // Try opening the file
47     FILE *imageFile;
48     imageFile=fopen("./anna.ppm","rb");
49     if(imageFile==NULL){
50         perror("ERROR: Cannot open output file");
51         exit(EXIT_FAILURE);
52     }
53
54     // Initialize empty image array
55     uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)
           +OFFSET);
56
57     // Read the image
58     fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+
           OFFSET, imageFile);
59
60     // Close the file
61     fclose(imageFile);
62
63     // Move the starting pointer and return the flattened image
           array
64     return image_array + OFFSET;
65 }
66
67 void save_image_array(uint8_t* image_array){
68     /*
69      * Save the data of an (RGB) image as a pixel map.
70      *
71      * Parameters:
72      *   - param1: The data of an (RGB) image as a 1D array
73      *
74      */
75     // Try opening the file
76     FILE *imageFile;
77     imageFile=fopen("./output_image_1.ppm","wb");
78     if(imageFile==NULL){
```

```
79          perror("ERROR: Cannot open output file");
80          exit(EXIT_FAILURE);
81      }
82
83      // Configure the file
84      fprintf(imageFile,"P6\n");                  // P6 filetype
85      fprintf(imageFile,"%d %d\n", M, N);         // dimensions
86      fprintf(imageFile,"255\n");                 // Max pixel
87
88      // Write the image
89      fwrite(image_array, 1, M*N*C, imageFile);
90
91      // Close the file
92      fclose(imageFile);
93  }
94
95  void write_to_csv(std::ofstream &csv_file, int blocks, int
        threads, int strides, float ms) {
96      csv_file << blocks << "," << threads << "," << strides << ","
            << ms << "\n";
97  }
98
99  void measure(uint8_t *d_a, uint8_t *d_r, std::ofstream &csv_file)
        {
100     int MAX_BLOCKS = 80;
101     int COUNT = 5;
102     int thread_sizes[4] = {16, 32, 48, 1024};
103     csv_file << "blocks,threads,strides,duration\n";
104
105
106     // Run function once before timing
107     invert<<<1, 1024>>>(d_a, d_r, M*N*C);
108
109     for (int j = 0; j < 4; j++) {
110         int THREADS = thread_sizes[j];
111
112         for (int BLOCKS = 1; BLOCKS <= MAX_BLOCKS; BLOCKS++) {
113             float total_ms = 0;
114
115             // take an average
116             for (int i = 0; i < COUNT; i++) {
117                 // Start Timing
118                 cudaEvent_t start_cuda, stop_cuda;
```

```
119                 cudaEventCreate(&start_cuda);
120                 cudaEventCreate(&stop_cuda);
121                 cudaEventRecord(start_cuda);
122 
123                 // Run the Device code
124                 invert<<<BLOCKS, THREADS>>>(d_a, d_r, M*N*C);
125 
126                 // Stop timing
127                 cudaEventRecord(stop_cuda);
128                 cudaEventSynchronize(stop_cuda);
129                 float ms;
130                 cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
131 
132                 total_ms += ms;
133             }
134 
135             float avg_ms = total_ms / COUNT;
136             int strides = (M*N*C) / (BLOCKS * THREADS);
137 
138             write_to_csv(csv_file, BLOCKS, THREADS, strides,
                    avg_ms);
139             std::cout << "Avg␣duration␣for␣" << BLOCKS << "␣
                    blocks,␣" << THREADS << "␣threads␣and␣equals␣" <<
                    strides << "␣strides:␣" << avg_ms << "␣ms" << std::
                    endl;
140         }
141     }
142 }
143 
144 
145 void measure_warp(uint8_t *d_a, uint8_t *d_r, std::ofstream &
    csv_file) {
146     int MAX_BLOCKS = 80;
147     int COUNT = 5;
148 
149     // Run function once before timing
150     invert<<<1, 1024>>>(d_a, d_r, M*N*C);
151 
152     csv_file << "blocks,threads,duration\n";
153 
154     int threads_1 = 32; // One warp
155     int threads_2 = 16; // Half a warp
156     int blocks_1 = 1;
```

```
157     int blocks_2 = 2;
158 
159     for (int j = 0; j < 15; j++) {
160         float total_ms_1 = 0;
161         float total_ms_2 = 0;
162 
163         std::cout << "Total␣amount␣of␣used␣threads␣(1):" << (
                threads_1 * blocks_1) << "␣(2):␣" << (threads_2 *
                blocks_2) << std::endl;
164 
165         // take an average
166         for (int i = 0; i < COUNT; i++) {
167             // Start Timing
168             cudaEvent_t start_cuda, stop_cuda;
169             cudaEventCreate(&start_cuda);
170             cudaEventCreate(&stop_cuda);
171             cudaEventRecord(start_cuda);
172 
173             // Run the Device code
174             invert<<<blocks_1, threads_1>>>(d_a, d_r, M*N*C);
175 
176             // Stop timing
177             cudaEventRecord(stop_cuda);
178             cudaEventSynchronize(stop_cuda);
179             float ms;
180             cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
181 
182             total_ms_1 += ms;
183         }
184 
185         // take an average
186         for (int i = 0; i < COUNT; i++) {
187             // Start Timing
188             cudaEvent_t start_cuda, stop_cuda;
189             cudaEventCreate(&start_cuda);
190             cudaEventCreate(&stop_cuda);
191             cudaEventRecord(start_cuda);
192 
193             // Run the Device code
194             invert<<<blocks_2, threads_2>>>(d_a, d_r, M*N*C);
195 
196             // Stop timing
197             cudaEventRecord(stop_cuda);
```

```
198             cudaEventSynchronize(stop_cuda);
199             float ms;
200             cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
201 
202             total_ms_2 += ms;
203         }
204 
205         float avg_ms_1 = total_ms_1 / COUNT;
206         float avg_ms_2 = total_ms_2 / COUNT;
207 
208         write_to_csv(csv_file, blocks_1, threads_1, (M*N*C) / (
               blocks_1 * threads_1), avg_ms_1);
209         write_to_csv(csv_file, blocks_2, threads_2, (M*N*C) / (
               blocks_2 * threads_2), avg_ms_2);
210         std::cout << "(1) Avg duration for " << blocks_1 << "
               blocks, " << threads_1 << " threads and equals " <<
               avg_ms_1 << " ms" << std::endl;
211         std::cout << "(2) Avg duration for " << blocks_2 << "
               blocks, " << threads_2 << " threads and equals " <<
               avg_ms_2 << " ms" << std::endl;
212 
213         if (threads_1 >= 1024){
214             blocks_1 = blocks_1 * 2;
215         } else {
216             threads_1 = threads_1 * 2;
217         }
218         blocks_2 = blocks_2 * 2;
219     }
220 }
221 
222 int main (void) {
223     uint8_t *d_a, *d_r;
224 
225     // Init csv
226     std::ofstream csv_file("data_ses3_1.csv");
227 
228     // Read the image
229     uint8_t* image_array = get_image_array();
230 
231     // Allocate output
232     uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);
233 
234     // Allocate Device memory
```

```
235     cudaMalloc((void**)&d_a, M*N*C * sizeof(uint8_t));
236     cudaMalloc((void**)&d_r, M*N*C * sizeof(uint8_t));
237 
238     // Change order of RGB values into new array, copy to Device
            memory
239     cudaMemcpy(d_a, image_array, M*N*C * sizeof(uint8_t),
            cudaMemcpyHostToDevice);
240 
241     measure_warp(d_a, d_r, csv_file);
242 
243     cudaMemcpy(new_image_array, d_r, M*N*C * sizeof(uint8_t),
            cudaMemcpyDeviceToHost);
244 
245     // Save the image
246     save_image_array(new_image_array);
247 
248     cudaFree(d_a);
249     cudaFree(d_r);
250 
251     csv_file.close();
252 
253     return 0;
254 }
```

5.3 Code Part 2

Listing 4: CUDA code for the unordered 1D array

```
1  /*
2   * Code snippet for importing / exporting image data.
3   *
4   * To convert an image to a pixel map, run 'convert <name>.<
        extension> <name>.ppm
5   *
6   */
7  #include <cstdint>      // Data types
8  #include <iostream>     // File operations
9  #include <fstream>
10 
11 // #define M 512        // Lenna width
12 // #define N 512        // Lenna height
13 #define M 960        // VR width
14 #define N 1280        // VR height
```

```
15 #define C 3          // Colors
16 #define OFFSET 16    // Header length
17 #define MAX_THREADS_PER_BLOCK 1024
18
19 __global__ void invert(uint8_t *a, uint8_t *r, int n) {
20     int idx = threadIdx.x + blockIdx.x * blockDim.x;
21     //int strides = n / MAX_THREADS_PER_BLOCK;
22     int strides = n / (blockDim.x * gridDim.x);
23
24
25     for (int i = 0; i < strides; i++) {
26         int array_index = idx + (i * blockDim.x);
27
28         if (array_index < n) {
29             // Lazy for the edge cases for the first two and last
                   two pixels
30             if (array_index % 3 == 0 && array_index > 6 &&
                  array_index < n - 6) {
31                 r[array_index] = 0.1*a[array_index - 6] +  0.25 *
                       a[array_index - 3] + 0.5 * a[array_index] +
                      0.25 * a[array_index + 3] + 0.1 * a[array_index
                       + 6];
32             } else {
33                 r[array_index] = 255 - a[array_index];
34             }
35         }
36     }
37 }
38
39 uint8_t* get_image_array(void){
40     /*
41      * Get the data of an (RGB) image as a 1D array.
42      *
43      * Returns: Flattened image array.
44      *
45      * Noets:
46      *  - Images data is flattened per color, column, row.
47      *  - The first 3 data elements are the RGB components
48      *  - The first 3*M data elements represent the firts row of
           the image
49      *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M},
           r_{1,0}, ..., b_{b,M}, ..., b_{N,M}
50      *
```

```
51      */
52     // Try opening the file
53     FILE *imageFile;
54     imageFile=fopen("./anna.ppm","rb");
55     if(imageFile==NULL){
56         perror("ERROR: Cannot open output file");
57         exit(EXIT_FAILURE);
58     }
59 
60     // Initialize empty image array
61     uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)
           +OFFSET);
62 
63     // Read the image
64     fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+
           OFFSET, imageFile);
65 
66     // Close the file
67     fclose(imageFile);
68 
69     // Move the starting pointer and return the flattened image
           array
70     return image_array + OFFSET;
71 }
72 
73 void save_image_array(uint8_t* image_array){
74     /*
75      * Save the data of an (RGB) image as a pixel map.
76      *
77      * Parameters:
78      *   - param1: The data of an (RGB) image as a 1D array
79      *
80      */
81     // Try opening the file
82     FILE *imageFile;
83     imageFile=fopen("./output_image_2.ppm","wb");
84     if(imageFile==NULL){
85         perror("ERROR: Cannot open output file");
86         exit(EXIT_FAILURE);
87     }
88 
89     // Configure the file
90     fprintf(imageFile,"P6\n");                // P6 filetype
```

```
91      fprintf(imageFile,"%d␣%d\n", M, N);       // dimensions
92      fprintf(imageFile,"255\n");               // Max pixel
93
94      // Write the image
95      fwrite(image_array, 1, M*N*C, imageFile);
96
97      // Close the file
98      fclose(imageFile);
99  }
100
101 void write_to_csv(std::ofstream &csv_file, int blocks, int
        threads, int strides, float ms) {
102     csv_file << blocks << "," << threads << "," << strides << ","
            << ms << "\n";
103 }
104
105 void measure(uint8_t *d_a, uint8_t *d_r, std::ofstream &csv_file)
        {
106     int MAX_BLOCKS = 120;
107     int COUNT = 5;
108     int thread_sizes[4] = {16, 32, 48, 1024};
109
110     // Run function once before timing
111     invert<<<1, 1024>>>(d_a, d_r, M*N*C);
112
113     for (int j = 0; j < 4; j++) {
114         int THREADS = thread_sizes[j];
115
116         for (int BLOCKS = 1; BLOCKS <= MAX_BLOCKS; BLOCKS++) {
117             float total_ms = 0;
118
119             // take an average
120             for (int i = 0; i < COUNT; i++) {
121                 // Start Timing
122                 cudaEvent_t start_cuda, stop_cuda;
123                 cudaEventCreate(&start_cuda);
124                 cudaEventCreate(&stop_cuda);
125                 cudaEventRecord(start_cuda);
126
127                 // Run the Device code
128                 invert<<<BLOCKS, THREADS>>>(d_a, d_r, M*N*C);
129
130                 // Stop timing
```

```
131                 cudaEventRecord(stop_cuda);
132                 cudaEventSynchronize(stop_cuda);
133                 float ms;
134                 cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
135 
136                 total_ms += ms;
137             }
138 
139             float avg_ms = total_ms / COUNT;
140             int strides = (M*N*C) / (BLOCKS * THREADS);
141 
142             write_to_csv(csv_file, BLOCKS, THREADS, strides,
                    avg_ms);
143             std::cout << "Avg␣duration␣for␣" << BLOCKS << "␣
                    blocks,␣" << THREADS << "␣threads␣and␣equals␣" <<
                    strides << "␣strides:␣" << avg_ms << "␣ms" << std::
                    endl;
144         }
145     }
146 }
147 
148 int main (void) {
149     uint8_t *d_a, *d_r;
150 
151     // Init csv
152     std::ofstream csv_file("data_ses3_2.csv");
153     csv_file << "blocks,threads,strides,duration\n";
154 
155     // Read the image
156     uint8_t* image_array = get_image_array();
157 
158     // Allocate output
159     uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);
160 
161     // Allocate Device memory
162     cudaMalloc((void**)&d_a, M*N*C * sizeof(uint8_t));
163     cudaMalloc((void**)&d_r, M*N*C * sizeof(uint8_t));
164 
165     // Change order of RGB values into new array, copy to Device
           memory
166     cudaMemcpy(d_a, image_array, M*N*C * sizeof(uint8_t),
           cudaMemcpyHostToDevice);
167 
```

```
168     measure(d_a, d_r, csv_file);
169
170     cudaMemcpy(new_image_array, d_r, M*N*C * sizeof(uint8_t),
            cudaMemcpyDeviceToHost);
171
172     // Save the image
173     save_image_array(new_image_array);
174
175     cudaFree(d_a);
176     cudaFree(d_r);
177
178     csv_file.close();
179
180     return 0;
181 }
```

Listing 5: CUDA code for the ordered 1D array

```
1  /*
2   * Code snippet for importing / exporting image data.
3   *
4   * To convert an image to a pixel map, run `convert <name>.<
        extension> <name>.ppm
5   *
6   */
7  #include <cstdint>      // Data types
8  #include <iostream>     // File operations
9  #include <fstream>
10
11 // #define M 512        // Lenna width
12 // #define N 512        // Lenna height
13 #define M 960         // VR width
14 #define N 1280         // VR height
15 #define C 3           // Colors
16 #define OFFSET 16     // Header length
17 #define MAX_THREADS_PER_BLOCK 1024
18
19 __global__ void invert(uint8_t *a, uint8_t *r, int n) {
20     int idx = threadIdx.x + blockIdx.x * blockDim.x;
21     int strides = n / (blockDim.x * gridDim.x);
22
23     for (int i = 0; i < strides; i++) {
24         int array_index = idx + (i * blockDim.x);
25
```

```
26          if (array_index > 2 && array_index < (n / 3) - 2) {
27              r[array_index] = 0.1*a[array_index - 2] +  0.25 * a[
                    array_index - 1] + 0.5 * a[array_index] + 0.25 * a[
                    array_index + 1] + 0.1 * a[array_index + 2];
28          } else {
29              r[array_index] = 255 - a[array_index];
30          }
31      }
32  }
33
34  uint8_t* get_image_array(void){
35      /*
36       * Get the data of an (RGB) image as a 1D array.
37       *
38       * Returns: Flattened image array.
39       *
40       * Noets:
41       *   - Images data is flattened per color, column, row.
42       *   - The first 3 data elements are the RGB components
43       *   - The first 3*M data elements represent the firts row of
                the image
44       *   - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M},
                r_{1,0}, ..., b_{b,M}, ..., b_{N,M}
45       *
46       */
47      // Try opening the file
48      FILE *imageFile;
49      imageFile=fopen("./anna.ppm","rb");
50      if(imageFile==NULL){
51          perror("ERROR: Cannot open output file");
52          exit(EXIT_FAILURE);
53      }
54
55      // Initialize empty image array
56      uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)
            +OFFSET);
57
58      // Read the image
59      fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+
            OFFSET, imageFile);
60
61      // Close the file
62      fclose(imageFile);
```

```
63
64      // Move the starting pointer and return the flattened image
            array
65      return image_array + OFFSET;
66  }
67
68  void save_image_array(uint8_t* image_array){
69      /*
70       * Save the data of an (RGB) image as a pixel map.
71       *
72       * Parameters:
73       *   - param1: The data of an (RGB) image as a 1D array
74       *
75       */
76      // Try opening the file
77      FILE *imageFile;
78      imageFile=fopen("./output_image_3.ppm","wb");
79      if(imageFile==NULL){
80          perror("ERROR: Cannot open output file");
81          exit(EXIT_FAILURE);
82      }
83
84      // Configure the file
85      fprintf(imageFile,"P6\n");                  // P6 filetype
86      fprintf(imageFile,"%d %d\n", M, N);         // dimensions
87      fprintf(imageFile,"255\n");                 // Max pixel
88
89      // Write the image
90      fwrite(image_array, 1, M*N*C, imageFile);
91
92      // Close the file
93      fclose(imageFile);
94  }
95
96  void order_rgb(uint8_t* input, uint8_t* output, int n) {
97      for (int i = 0; i < n / 3; i++) {
98          output[i] = input[i * 3];
99          output[(n / 3) + i] = input[i * 3 + 1];
100         output[2 * (n / 3) + i] = input[i * 3 + 2];
101     }
102 }
103
104
```

```
105 void reorder_rgb(uint8_t* input, uint8_t* output, int n) {
106     for (int i = 0; i < n / 3; i++) {
107         output[i * 3] = input[i];
108         output[i * 3 + 1] = input[(n / 3) + i];
109         output[i * 3 + 2] = input[2 * (n / 3) + i];
110     }
111 }
112 
113 void write_to_csv(std::ofstream &csv_file, int blocks, int
       threads, int strides, float ms) {
114     csv_file << blocks << "," << threads << "," << strides << ","
            << ms << "\n";
115 }
116 
117 void measure(uint8_t *d_a, uint8_t *d_r, std::ofstream &csv_file)
        {
118     int MAX_BLOCKS = 120;
119     int COUNT = 5;
120     int thread_sizes[4] = {16, 32, 48, 1024};
121 
122     // Run function once before timing
123     invert<<<1, 1024>>>(d_a, d_r, M*N*C);
124 
125     for (int j = 0; j < 4; j++) {
126         int THREADS = thread_sizes[j];
127 
128         for (int BLOCKS = 1; BLOCKS <= MAX_BLOCKS; BLOCKS++) {
129             float total_ms = 0;
130 
131             // take an average
132             for (int i = 0; i < COUNT; i++) {
133                 // Start Timing
134                 cudaEvent_t start_cuda, stop_cuda;
135                 cudaEventCreate(&start_cuda);
136                 cudaEventCreate(&stop_cuda);
137                 cudaEventRecord(start_cuda);
138 
139                 // Run the Device code
140                 invert<<<BLOCKS, THREADS>>>(d_a, d_r, M*N*C);
141 
142                 // Stop timing
143                 cudaEventRecord(stop_cuda);
144                 cudaEventSynchronize(stop_cuda);
```

```
145                 float ms;
146                 cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
147 
148                 total_ms += ms;
149             }
150 
151             float avg_ms = total_ms / COUNT;
152             int strides = (M*N*C) / (BLOCKS * THREADS);
153 
154             write_to_csv(csv_file, BLOCKS, THREADS, strides,
                    avg_ms);
155             std::cout << "Avg␣duration␣for␣" << BLOCKS << "␣
                    blocks,␣" << THREADS << "␣threads␣and␣equals␣" <<
                    strides << "␣strides:␣" << avg_ms << "␣ms" << std::
                    endl;
156         }
157     }
158 }
159 
160 int main (void) {
161     uint8_t *d_a, *d_r;
162 
163     // Init csv
164     std::ofstream csv_file("data_ses3_3.csv");
165     csv_file << "blocks,threads,strides,duration\n";
166 
167     // Read the image
168     uint8_t* image_array = get_image_array();
169 
170     // Allocate output
171     uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);
172     uint8_t* tmp = (uint8_t*)malloc(M*N*C);
173 
174     // Allocate Device memory
175     cudaMalloc((void**)&d_a, M*N*C * sizeof(uint8_t));
176     cudaMalloc((void**)&d_r, M*N*C * sizeof(uint8_t));
177 
178     // Change order of RGB values into new array, copy to Device
           memory
179     order_rgb(image_array, tmp, N*M*C);
180     cudaMemcpy(d_a, tmp, M*N*C * sizeof(uint8_t),
           cudaMemcpyHostToDevice);
181 
```

```
182     measure(d_a, d_r, csv_file);
183
184     cudaMemcpy(tmp, d_r, M*N*C * sizeof(uint8_t),
            cudaMemcpyDeviceToHost);
185
186     // Reorder the the array back to it's original layout
187     reorder_rgb(tmp, new_image_array, N*M*C);
188
189     // Save the image
190     save_image_array(new_image_array);
191
192     cudaFree(d_a);
193     cudaFree(d_r);
194
195     csv_file.close();
196
197     return 0;
198 }
```